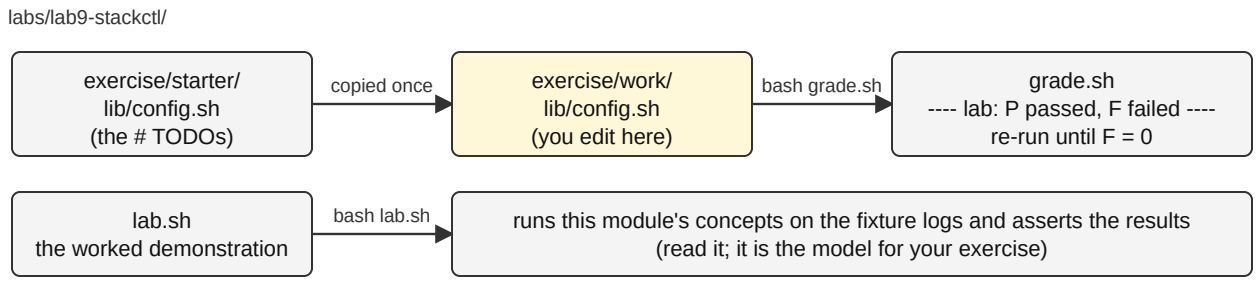
Lab 9 — stackctl: Operate the Program's Project from Bash (LAB-09a, 09b, 09c) (Instructor Version)

Instructor Note: Gray boxes and answers marked in red appear in the instructor copy only.

Topology



Environment Table

All paths are relative to `labs/lab9-stackctl/exercise/`. The tool you finish is a small infrastructure-as-code command, `stackctl`, that runs the SecMLOps project — **fraud-clf**, the fraud-detection API Course 2 takes to production — as two services per environment: `serve` (the FastAPI model service, built from the inherited copy in `app/`) behind `gateway` (nginx, the only published port). Two environments: `test` on port 18080 and `prod` on 18081.

© 2026 SecMLOps · Course 0 — Bash Scripting. Course material; not for redistribution outside the course.

File	What it is	You edit it?
`TASK.md`	The statement of the three labs	No
`starter/`	The whole tool with seven `# TODO` functions	No — the grader copies it
`work/`	**Your** copy (created by the first `grade.sh` run)	**Yes**, the files below
`work/bin/stackctl`	The dispatcher: `stackctl [-e ENV] COMMAND`; exit 0 ok · 1 runtime · 2 usage/validation · 3 check failed	No
`work/lib/driver.sh`	`drv_compose` and `drv_http_get`; with `STACKCTL_DRIVER=mock` it records calls instead of running Docker	No
`work/lib/config.sh`	`parse_conf`, `validate_conf` (**LAB-09a**); given: `CONF_KEYS`, `is_known_key`, `load_conf`, `render_compose`	**Yes**
`work/lib/ops.sh`	`cmd_check` (**09a**); `with_lock`, `cmd_plan`, `cmd_up`, `cmd_destroy` (**09b**); given: `applied_dir`, `running_services`, `cmd_status`, `cmd_logs`	**Yes**
`work/compose/*.tmpl`	The compose and nginx templates `render_compose` fills	No
`work/envs/test.conf`, `prod.conf`	Environment files: `PORT`, `PROJECT`, `APP_DIR`, `IMAGE_SERVE`, `IMAGE_GATEWAY`, `HEALTH_PATH`, `HEALTH_BODY`, `WAIT_TIMEOUT`	Only to append a digest in 09c
`work/app/`	The inherited fraud-clf baseline plus `Dockerfile.course`; read `COURSE0-NOTE.md`	**Never**
`work/tests/run.sh`, `assert.sh`	The tool's own suite (mock driver): `# N passed, M failed`	No
`work/tests/live-smoke.sh`	Build → up → check → predict → converge → destroy on a real daemon	No
`work/tools/record-manifest.sh`	Prints the gateway digest and serve image ID from `docker image inspect`	No
`work/RUNBOOK.md`	The handover template, six sections with `TODO`s (**LAB-09c**)	**Yes**
`work/REFLECTION.md`	Your answers to R1 and R2	**Yes** (you create it)
`grade.sh`	The grader: 34 checks in sections `09a`, `09b`, `09c`, run with the mock driver	No

Objectives

- **Part 1:** Prepare the exercise folder and read the starter's report.
- **Part 2 (LAB-09a):** Configuration is data — `parse_conf`, `validate_conf`, `cmd_check`.
- **Part 3 (LAB-09b):** Plan, apply, destroy, lock — `with_lock`, `cmd_plan`, `cmd_up`, `cmd_destroy`.
- **Part 4:** Verify with the grader and the tool's own suite; write the two reflections.
- **Part 5 (LAB-09c):** Break each environment on purpose, fill `RUNBOOK.md`, and on a Docker machine record the manifest.

© 2026 SecMLOps · Course 0 — Bash Scripting. Course material; not for redistribution outside the course.

Background / Scenario

Everything in M00–M08 was about one script. This lab is about a *tool that operates infrastructure*, and the rules change in three ways. The environment file is **data**: it is parsed line by line against a whitelist, never `sourced`, so `PORT=18080; rm -rf /` is a validation error and not a command. **Plan never mutates**: `plan` renders the desired state into a temporary directory and compares it with what was last applied; only `up` writes the applied snapshot and calls `compose up`. **Destruction needs the environment's name**: `destroy` refuses unless `CONFIRM_DESTROY=<env>` matches `-e <env>`, runs `down -v` scoped to one project, and never prunes.

You can do LAB-09a and 09b without Docker: the mock driver records every `compose` call in a log the grader reads. The live smoke test and the manifest in LAB-09c need a daemon, and those are the only evidence that the stack really builds and provisions (CAP-01 asks for them).

Required Resources

- A shell where `bash tools-check.sh` reports `ready` (with `shellcheck` and `flock`). Docker only for Part 5, step 4.
- Module M09, steps S09.2–S09.5; the Compose documentation; AI assistance for lookups (disclose it). Do **not** open `solution/`.

> **Instructor Note:** No fixed duration; learners typically need about an hour each for 09a and 09b and a third sitting for 09c. Grading ≈ 10 minutes per learner for 09a/b and ≈ 10 for the runbook. Observed: the starter scores `---- lab9: 10 passed, 24 failed ----`; the reference 34/34, and `tests/run.sh` with the reference prints `# 34 passed, 0 failed`.
>
> Two things to say to the cohort before they start. First, `work/app/` is operated, not fixed — a "repaired" `app/` is Not met for 09c. Second, the **digest rule**: no digest or image ID may ever be typed from a document; it must come from `docker image inspect` on the learner's own machine, or section 5 says "pending". Compare any gateway digest a learner records with your own V-04 value for the same architecture.

Part 1: Prepare the exercise folder

Step 1: Run the grader once.

```
cd labs/lab9-stackctl/exercise
bash grade.sh
```

Expected: `---- lab9: 10 passed, 24 failed ----`. The passing lines are the given parts (the dispatcher, the driver, lint); every `# TODO` function fails.

Step 2: Read the tool before editing it.

Read `work/app/COURSE0-NOTE.md`, then `work/bin/stackctl` (the dispatch `case` at the end: `plan`, `check`, `status`, `logs` run `cmd_validate` first; `up` and `destroy` run under `with_lock`), then the given helpers in `work/lib/config.sh` (`render_compose`: the bind-mount path inside the rendered file always points at `.render/<env>` and the build context at `$ROOT/<APP_DIR>`, whichever directory you render *into*

© 2026 SecMLOps · Course 0 — Bash Scripting. Course material; not for redistribution outside the course.

— that is what makes a temporary render comparable with the applied one). Each `# TODO` lists the exact calls to make.

Part 2 (LAB-09a): Configuration is data

Step 1: `parse_conf` in `work/lib/config.sh`.

Read the file line by line; strip comments and trailing whitespace; skip blank lines; require `KEY=VALUE`; reject a key that is not upper-case or not in `CONF_KEYS`; store into the associative array `CONF`. Return 2 on the first bad line, with the file and line number in the message.

```
parse_conf() {
  local file="$1" line key val n=0
  [[ -r "$file" ]] || { err "cannot read environment file: $file"; return 2; }
  while IFS= read -r line || [[ -n "$line" ]]; do
    n=$(( n + 1 ))
    line="${line%%#*}"
    line="${line%"${line##*[![:space:]]}"}"
    [[ -z "$line" ]] && continue
    [[ "$line" == *=* ]] || { err "$file:$n: expected KEY=VALUE, got: $line"; return 2;
}
    key="${line%%=*}"; val="${line#*=}"
    [[ "$key" =~ ^[A-Z][A-Z_]*$ ]] || { err "$file:$n: bad key name: $key"; return 2; }
    is_known_key "$key" || { err "$file:$n: unknown key: $key (allowed: ${CONF_KEYS
[*]})"; return 2; }
    CONF["$key"]="$val"
  done < "$file"
}
```

How it works: this is S09.2's loop. `${line%%#*}` removes everything from the first `#`; the next line removes trailing whitespace (the inner `${line##*[![:space:]]}` is "the trailing whitespace", and `%` strips it). `${line%%=*}` is the text before the first `=`, `${line#*=}` the text after it — so a value may itself contain `=` or `;` and it stays a *value*. Nothing here executes the line.

Step 2: `validate_conf`.

A sequence of `[[ … ]] || { err "…"; return 2; }` lines, in this order: every key in `CONF_KEYS` present and non-empty; `PORT` digits only **and then** 1024–65535 (test the shape before the numeric comparison, or a non-number aborts the arithmetic); `PROJECT` `^[a-z][a-z0-9-]*$`; `WAIT_TIMEOUT` digits; the two images `name:tag` optionally `@sha256:<64 hex>`; `APP_DIR` a relative path with no spaces or `..`, and `"$ROOT/${CONF[APP_DIR]}/Dockerfile.course"` must exist; `HEALTH_PATH` starts with `/` and has no spaces; `HEALTH_BODY` one line.

© 2026 SecMLOps · Course 0 — Bash Scripting. Course material; not for redistribution outside the course.

Instructor Note:

```
validate_conf() {
  local k
  for k in "${CONF_KEYS[@]}"; do
    [[ -n "${CONF[$k]:-}" ]] || { err "$CONF_FILE: missing required key: $k"; return
2; }
  done
  [[ "${CONF[PORT]}" =~ ^[0-9]+$ ]] || { err "PORT must be a number, got: ${CONF[POR
T]}"; return 2; }
  (( CONF[PORT] >= 1024 && CONF[PORT] <= 65535 )) || { err "PORT must be 1024–65535,
got: ${CONF[PORT]}"; return 2; }
  [[ "${CONF[PROJECT]}" =~ ^[a-z][a-z0-9-]*$ ]] || { err "PROJECT must be lowercase
letters, digits, dashes, got: ${CONF[PROJECT]}"; return 2; }
  [[ "${CONF[WAIT_TIMEOUT]}" =~ ^[0-9]+$ ]] || { err "WAIT_TIMEOUT must be a number
of seconds"; return 2; }
  for k in IMAGE_SERVE IMAGE_GATEWAY; do
    [[ "${CONF[$k]}" =~ ^[a-z0-9][a-z0-9._/-]*:[A-Za-z0-9._-]+(@sha256:[0-9a-f]{6
4})?$ ]] \
      || { err "$k must be name:tag or name:tag@sha256:<64 hex>, got: ${CONF[$k]}";
return 2; }
  done
  [[ "${CONF[APP_DIR]}" =~ ^[A-Za-z0-9._-]+(/[A-Za-z0-9._-]+)*$ ]] \
    || { err "APP_DIR must be a relative path inside the tool (no spaces, no ..), go
t: ${CONF[APP_DIR]}"; return 2; }
  [[ -f "$ROOT/${CONF[APP_DIR]}/Dockerfile.course" ]] \
    || { err "APP_DIR ${CONF[APP_DIR]}/ has no Dockerfile.course — is the fraud-clf
app copy in place?"; return 2; }
  [[ "${CONF[HEALTH_PATH]}" =~ ^/[A-Za-z0-9._/-]*$ ]] || { err "HEALTH_PATH must sta
rt with / and contain no spaces, got: ${CONF[HEALTH_PATH]}"; return 2; }
  [[ "${CONF[HEALTH_BODY]}" != *[$'\n\r']* ]] || { err "HEALTH_BODY must be one lin
e"; return 2; }
}
```

A `case`-based version passes. Messages without the file and line are acceptable — mention them for the runbook.

Step 3: `cmd_check` in `work/lib/ops.sh`.

Build the URL `http://127.0.0.1:${CONF[PORT]}${CONF[HEALTH_PATH]}`, fetch it through the gateway with `drv_http_get`, and compare the body with `HEALTH_BODY` **byte for byte**: unreachable → message and `return 1`; different → show got/expected and `return 3`; equal → one line and 0.

```
cmd_check() {
  local url="http://127.0.0.1:${CONF[PORT]}${CONF[HEALTH_PATH]}" got
  got="$(drv_http_get "$url" && printf x)" \
    || { err "check: $url unreachable (is the stack up? try: $PROG -e $ENV_NAME statu
s)"; return 1; }
  got="${got%x}"
  [[ "$got" == "${CONF[HEALTH_BODY]}" ]] \
    || { err "check: $url returned [$got], expected [${CONF[HEALTH_BODY]}]"; return 3; }
  info "check: $url → [$got] ok"
}
```

Note: `$( )` strips trailing newlines, so a body of `{"status":"ok"}` followed by a newline would look identical to the correct one. The `&& printf x` appends a sentinel that protects the real last byte;

© 2026 SecMLOps · Course 0 — Bash Scripting. Course material; not for redistribution outside the course.

`${got%x}` removes it again. The grader's check `check: body + trailing newline → 3` exists for exactly this.

Step 4: Try it by hand (mock driver).

```
export STACKCTL_DRIVER=mock STACKCTL_MOCK_STATE=/tmp/m09; mkdir -p /tmp/m09
bash work/bin/stackctl -e test validate; echo "status=$?"
sed 's/^PORT=.*/PORT=80a8/' work/envs/test.conf > work/envs/bad.conf
bash work/bin/stackctl -e bad validate; echo "status=$?"
STACKCTL_MOCK_HEALTH='{"status":"ok"}' bash work/bin/stackctl -e test check; echo "statu
s=$?"
STACKCTL_MOCK_HEALTH='{"status":"nope"}' bash work/bin/stackctl -e test check; echo "sta
tus=$?"
rm -f work/envs/bad.conf
```

Expected: `status=0`; `PORT must be a number, got: 80a8` and `status=2`; `check: http://127.0.0.1:18080/api/health → [{"status":"ok"}] ok` and `status=0`; `check: … returned [{"status":"nope"}], expected [{"status":"ok"}]` and `status=3`.

At this point `bash grade.sh` should show all fourteen `09a` lines green: `validate test → 0`; `the conf file is parsed, never executed` (the grader plants `HEALTH_BODY=ok; touch …/pwned` and checks the file was **not** created); `PORT=80a8 → 2`; `PORT=70000 → 2`; `missing HEALTH_BODY → 2`; `unknown key COLOUR → 2`; `APP_DIR without Dockerfile.course → 2`; `no source/eval of the conf anywhere`; the three `check` codes 0/3/3; `check: unreachable → 1`; `check asks the gateway for HEALTH_PATH`.

Part 3 (LAB-09b): Plan, apply, destroy, lock

Step 1: `with_lock`.

Three lines: open file descriptor 9 on `envs/<env>.lock`, take a **non-blocking** lock, run the command that was passed in.

```
with_lock() {
  local lock="$ROOT/envs/$ENV_NAME.lock"
  exec 9>"$lock"
  flock -n 9 || { err "$ENV_NAME is locked by another stackctl run ($lock); try again la
ter"; return 1; }
  "$@"
}
```

`-n` matters: a blocking `flock` would make a second operator wait silently instead of failing fast with exit 1 (`up while test is locked → 1`). The lock is per environment, so an operator on `prod` is never blocked by `test`.

Step 2: `cmd_plan` — never mutates.

Render the desired state into `$WORK/plan` (the per-run `mktemp` directory the dispatcher removes on exit), compare **both** rendered files with the applied snapshot in `.render/<env>/`, count running services, print exactly one `up would:` line and a `limits` line. No `compose up` or `down` call.

© 2026 SecMLOps · Course 0 — Bash Scripting. Course material; not for redistribution outside the course.

Instructor Note:

```
cmd_plan() {
  local applied; applied="$(applied_dir)"
  render_compose "$WORK/plan"
  info "plan for $ENV_NAME (project ${CONF[PROJECT]}):"
  info "  desired : port ${CONF[PORT]}, serve ${CONF[IMAGE_SERVE]} (from ${CONF[APP_
DIR]}/), gateway ${CONF[IMAGE_GATEWAY]}, health GET ${CONF[HEALTH_PATH]} = '${CONF[H
EALTH_BODY]}'"
  local drift="none"
  if [[ ! -f "$applied/stack.yml" ]]; then
    info "  applied : never applied in this checkout"; drift="new"
  elif ! diff -q "$applied/stack.yml" "$WORK/plan/stack.yml" >/dev/null \
       || ! diff -q "$applied/gateway.conf" "$WORK/plan/gateway.conf" >/dev/null; th
en
    info "  applied : differs from desired:"
    diff "$applied/stack.yml" "$WORK/plan/stack.yml" | sed 's/^/            /' || tr
ue
    diff "$applied/gateway.conf" "$WORK/plan/gateway.conf" | sed 's/^/            /'
|| true
    drift="changed"
  else
    info "  applied : identical to desired"
  fi
  local n; n="$(running_services "$applied/stack.yml" | grep -c . || true)"
  info "  running : $n of 2 services (from compose ps)"
  if   [[ "$drift" == new ]];     then info "  up would: create project ${CONF[PROJE
CT]} (2 services)"
  elif [[ "$drift" == changed ]]; then info "  up would: re-render and recreate the
services whose definition changed"
  elif (( n < 2 ));               then info "  up would: start the $(( 2 - n )) stop
ped service(s); no definition changes"
  else                                 info "  up would: nothing — converged"
  fi
  info "  limits  : plan cannot predict the image build, pulls or health outcomes; o
nly 'up' proves those"
}
```

The grader greps for exactly one line starting `up would:` and, after a second `up`, for `up would: nothing`. **A trap it cannot see:** a `plan` that renders into `.render/` and copies the result to `$WORK/plan` passes 34/34 — but on a fresh checkout it reports `applied : identical` and `up would: start the 2 stopped service(s)` instead of `create`, because `plan` has just written the applied snapshot. Read every learner's `cmd_plan`; that is R1's point.

Step 3: `cmd_up`.

Render into the **applied** directory, then `compose up -d --wait --wait-timeout <WAIT_TIMEOUT>`; on failure point the operator at `logs` and return 1; on success say how many of the two services run.

```
cmd_up() {
  local applied; applied="$(applied_dir)"
  render_compose "$applied"
  info "up: $ENV_NAME (project ${CONF[PROJECT]}, port ${CONF[PORT]}, wait ≤ ${CONF[WAIT_
TIMEOUT]}s)"
  drv_compose "${CONF[PROJECT]}" "$applied/stack.yml" up -d --wait --wait-timeout "${CON
F[WAIT_TIMEOUT]}" \
```

© 2026 SecMLOps · Course 0 — Bash Scripting. Course material; not for redistribution outside the course.

```
    || { err "up failed for $ENV_NAME; see: $PROG -e $ENV_NAME logs"; return 1; }
  info "up: done — $(running_services "$applied/stack.yml" | grep -c . || true) of 2 ser
vices running"
}
```

The grader looks for exactly `up -d --wait --wait-timeout 120` in the mock call log (`test.conf` sets `WAIT_TIMEOUT=120`). The dispatcher already runs `cmd_validate` before `up`, which is why `up with an invalid conf → 2` **and made no up call**.

Step 4: `cmd_destroy`.

Refuse with 2 unless `CONFIRM_DESTROY` equals **this** environment's name; then `down -v` scoped to this project; remove the applied directory; print one line. Never `docker system prune`.

```
cmd_destroy() {
  local applied; applied="$(applied_dir)"
  [[ "${CONFIRM_DESTROY:-}" == "$ENV_NAME" ]] \
    || { err "destroy refused: set CONFIRM_DESTROY=$ENV_NAME to destroy project ${CONF[P
ROJECT]}"; return 2; }
  render_compose "$WORK/destroy"
  local file="$applied/stack.yml"; [[ -f "$file" ]] || file="$WORK/destroy/stack.yml"
  drv_compose "${CONF[PROJECT]}" "$file" down -v || { err "destroy failed for ${CONF[PRO
JECT]}"; return 1; }
  rm -rf "$applied"
  info "destroy: project ${CONF[PROJECT]} removed (containers, network, volumes)"
}
```

The temporary render is the fallback compose file for a project that was never applied in this checkout but may still exist in the daemon.

Step 5: Try the whole cycle by hand (mock driver).

```
rm -rf /tmp/m09 work/.render; mkdir -p /tmp/m09
S="bash work/bin/stackctl -e test"
$S plan | grep 'up would'
$S up; $S plan | grep 'up would'
$S destroy; echo "status=$?"
CONFIRM_DESTROY=prod $S destroy; echo "status=$?"
CONFIRM_DESTROY=test $S destroy; echo "status=$?"
cat /tmp/m09/calls.log
```

Expected: `up would: create project fraudclf-test (2 services)`; after `up`, `up would: nothing — converged`; `destroy refused: set CONFIRM_DESTROY=test …` with `status=2` twice; then `status=0`; and a call log whose only mutations are one `up -d --wait --wait-timeout 120` line and one `down -v` line for project `fraudclf-test` (the `config -q` and `ps --services --status running` lines are the read-only calls `validate` and `plan` make); nothing with `prune`.

Part 4: Verify and reflect

Step 1: Grader and the tool's own suite.

```
bash grade.sh
bash work/tests/run.sh | tail -1
diff -r work/app starter/app && echo "app untouched"
```

© 2026 SecMLOps · Course 0 — Bash Scripting. Course material; not for redistribution outside the course.

Expected: `---- lab9: 32 passed, 2 failed ----` until LAB-09c (the two `09c` lines are the runbook's), `# 34 passed, 0 failed`, and `app untouched`.

Symptom	Cause	Fix
`the conf file is parsed, never executed` fails	`source`/`eval`/`.` on the file somewhere	Parse it; S09.2
`check: body + trailing newline → 3` fails	Plain `$( )` capture strips the newline	The `&& printf x` sentinel
`plan made no up/down call` fails	`cmd_plan` calls `cmd_up` "to see"	Render to `$WORK/plan`, diff, print
`up while test is locked → 1` fails after a ≈3 s pause	`flock` without `-n` blocks	`flock -n 9`
`destroy with CONFIRM_DESTROY=prod on test → 2` fails	Only `-n "$CONFIRM_DESTROY"` is tested	Compare with `"$ENV_NAME"`
`up waits for health` fails	`--wait-timeout` missing or hard-coded	Pass `"${CONF[WAIT_TIMEOUT]}"`

Step 2: Reflection.

Create `work/REFLECTION.md` (3–5 sentences each):

R1. `plan` renders the compose file into a temporary directory and `up` renders it into `.render/test/`. Why two places, and what would break if `plan` wrote to `.render/`?

> **Answer: Met:** `plan` must be pure, so it renders into the per-run `mktemp` directory (`$WORK`, removed by the dispatcher's trap) and only *reads* `.render/test/`; `up` renders there because that file is the record of what was applied and the bind-mount source the containers use. If `plan` wrote there, every `plan` would overwrite the applied snapshot, drift could never be detected and `plan` would have become a mutation. **Strong:** adds that the bind-mount and build-context paths inside the file are written as `.render/test` and `$ROOT/app` even for the temporary render so the two compare equal, or names the consequence for a running stack (the mounted `gateway.conf` would change under a live container). **Not met:** "to keep things tidy", or no mention of drift.

R2. The grader accepts `CONFIRM_DESTROY=test` for `test` and rejects `CONFIRM_DESTROY=prod` for `test`. Explain why a confirmation that accepted *any* non-empty value would be worse than no confirmation at all.

© 2026 SecMLOps · Course 0 — Bash Scripting. Course material; not for redistribution outside the course.

> **Answer:** **Met:** a confirmation that accepts any value tells the operator "you confirmed" while confirming nothing — it trains the habit of typing the magic word without reading which environment is selected; requiring the environment's name makes the confirmation carry the one fact that matters (`-e test` + `CONFIRM_DESTROY=test`). With no confirmation at all, at least nobody believes a safeguard exists. **Strong:** connects it to the lock (both force a human decision at the point of risk), to Course 2's named-environment approval before promotion, or to CI pipelines copy-pasting commands so a generic `yes` propagates. **Not met:** "it is more secure" without the mismatch mechanism.
>
> *Calibration.* Met: "If `CONFIRM_DESTROY=yes` worked everywhere, I'd put it in my shell profile and never think again; requiring `test` or `prod` means I have to say which one I'm destroying, and it must match `-e`. No confirmation would at least not pretend." Not met: "Because a wrong value should be rejected."

Part 5 (LAB-09c): Diagnose and hand over

Step 1: Break each environment on purpose, one fault at a time.

For each fault record: the command, the **first line** of output, the exit status, the fix. Use a copy of the conf (`cp work/envs/test.conf work/envs/t2.conf`, `-e t2`) and the mock driver.

Fault	Command	What to look for
`PORT=180a80`	`stackctl -e t2 validate`	`PORT must be a number`, exit 2
The `HEALTH_BODY` line deleted	`validate`	`missing required key: HEALTH_BODY`, exit 2
`HEALTH_BODY={"status": "ok"}` (a space) with `STACKCTL_MOCK_HEALTH='{"status":"ok"}'`	`check`	`returned […], expected […]`, exit 3
`HEALTH_PATH=/health`	`check`	passes validation; the mock answers the gateway path — read what `check` prints and decide what a real app would return
A held lock (S09.5: `flock work/envs/test.lock sleep 30 &`)	`up`	`locked by another stackctl run`, exit 1, immediately
On a daemon: a port already in use	`up`	`up failed … see: stackctl -e test logs`, exit 1

Step 2: Write sections 1–4 of `work/RUNBOOK.md`.

Section 1 describes the stack (two services, two environments, which port is published); section 2 the normal order of commands (`validate` → `plan` → `up` → `check` → `plan` again → `destroy` with the name); sections 3 and 4 are your table above, as a decision tree keyed on **exit codes** (2 → the conf; `up` 1 → `logs`, port, image, timeout; `check` 3 → body; `check` 1 → down or port drift; 1 with `locked` → another operator or a stale holder), and the recovery that rebuilds **one** environment with `CONFIRM_DESTROY`. Replace every `TODO`; keep the six headings.

© 2026 SecMLOps · Course 0 — Bash Scripting. Course material; not for redistribution outside the course.

Step 3: Section 6 — inherited findings.

Read `work/app/config.py`, `requirements.txt`, `src/serve.py` and `Dockerfile.course` once, as the baseline README asks. Write what worries you and which Course 2 milestone addresses it. Do **not** change `app/`.

> **Instructor Note:** The five findings the reference lists, with their milestones: fake S3 keys in `config.py` → M2 (OpenBao); `requests==2.19.1` → M4 (SCA gate); unauthenticated `/api/predict` → M10 (zero-trust proxy); unsigned `model.pkl` → M6; the image runs as root → M7. Strong answers add one more (the health endpoint is unauthenticated too; training data is regenerated at build so two builds differ).

Step 4: On a machine with a Docker daemon — the live evidence.

```
bash work/tests/live-smoke.sh test
bash work/tools/record-manifest.sh
```

The smoke test builds the serve image, brings the stack up, checks health, sends a prediction, proves a second `up` converges, and destroys — leaving nothing behind. The first run takes a few minutes (pip installs; the model trains). Paste `record-manifest.sh`'s output into section 5 and append `@sha256:…` to `IMAGE_GATEWAY` in `work/envs/test.conf` from **that output**. Without a daemon, write in section 5 that the manifest is pending and leave the key tag-pinned. **Never type a digest or an image ID from a document** — this manual included.

Step 5: Final grade.

```
bash grade.sh | tail -1
```

Expected: `---- lab9: 34 passed, 0 failed ----`.

Hand-in and grading

LAB-09a/b: `work/lib/config.sh`, `work/lib/ops.sh`, `work/REFLECTION.md`, the grader output. **Pass** = every `09a` and `09b` check green, `# 34 passed` from the tool's suite, an untouched `app/`, and both reflections at Met or better. Unlimited attempts; this is practice evidence for CAP-01, not CAP-01 itself.

LAB-09c: `work/RUNBOOK.md` with no `TODO`, and — when you have a daemon — `work/MANIFEST.txt` plus the `live-smoke.sh` transcript. **Pass** = the two `09c` checks green and every value in section 5 traceable to `docker image inspect` on your machine.

© 2026 SecMLOps · Course 0 — Bash Scripting. Course material; not for redistribution outside the course.

Instructor Note: Grading LAB-09a/b (≈10 min per learner).

1. `GRADE_TARGET=/path/to/their/work bash labs/lab9-stackctl/exercise/grade.sh` → every `09a` and `09b` line PASS.
2. `bash /path/to/their/work/tests/run.sh | tail -1` → `# 34 passed, 0 failed`.
3. `diff -r /path/to/their/work/app starter/app` → **no output**.
4. Read `cmd_plan` (the `.render/` trap above) and `with_lock` (`-n`); read the reflections against the descriptors.
5. Record `LAB-09a/b PASS` only with all of the above.

Acceptable variations: a `case`-based `validate_conf`; different `plan` wording as long as exactly one `up would:` line exists and the converged line contains `up would: nothing`; `cmd_check` with `read -r -d ''` to keep bytes; another fd number in `with_lock`. **Deviations the grader catches (observed):** blocking `flock` → 33/34 after a ≈3 s pause; `CONFIRM_DESTROY` tested only for non-empty → 31/34 (the wrong-env refusal, the no-down-call check and the scoped-down check fail); `cmd_up` without `--wait-timeout`; `cmd_check` ignoring `HEALTH_PATH`. **Deviations it cannot see:** the `.render/` plan trap (passes 34/34 — ask); `exit` instead of `return` inside functions (passes; makes the functions impossible to unit-test by sourcing — mention); validation messages without file and line.

Model feedback (Not yet): "`plan made no up/down call` FAIL: your `cmd_plan` calls `cmd_up` to 'see what would happen' — a plan that applies is not a plan. Render to `$WORK/plan` and compare with `.render/test/`." Remediation: S09.3, then S09.4.

Equivalent reassessment: same grader; **R1′** "Why does `up` run `validate` even though you just ran `validate` yourself?" (Met: the file may have changed in between; every mutation is preceded by its own validation) and **R2′** "Why is the lock *per environment* rather than one lock for the whole tool?" (Met: `test` and `prod` are isolated by design — projects, ports, volumes — so an operator on `prod` must not be blocked by `test`).

Reviewing the runbook, LAB-09c (≈10 min). The reference `solution/RUNBOOK.md` is the model. *Met:* every symptom in the template has a command, an observed first line and a fix; no `TODO`; section 5 holds a pasted `record-manifest.sh` output **or** states the manifest is pending with the key tag-pinned; section 6 names at least three findings with the right milestone. *Strong:* adds a symptom the template did not list (`plan` shows drift after a teammate edited the conf; `check` exits 1 after a `PORT` change without `up`; `plan` says converged after `app/` changed) with its diagnosis, or a finding beyond the five. *Not met:* generic prose ("check the logs"), a symptom with no command, a digest or image ID with no live transcript, or a "fixed" `app/`. **Critical regardless of the rest:** any value in section 5 or `envs/*.conf` the learner cannot show coming from `docker image inspect` on their machine — ask for the `live-smoke.sh` transcript and compare the gateway digest with your V-04 value for the same architecture.

Evidence handling / reset: collect `work/lib/config.sh`, `work/lib/ops.sh`, `work/REFLECTION.md`, `work/RUNBOOK.md`, the grader output and, where present, `MANIFEST.txt` and the smoke transcript. Between learners on a shared machine: `rm -rf work /tmp/lab9-grade.* /tmp/m09; rm -f work/envs/*.lock`.

© 2026 SecMLOps · Course 0 — Bash Scripting. Course material; not for redistribution outside the course.